

Zomato Case Study — Answer Key

Questions from Class

1. Select a particular database

```
USE zomato;
```

2. Count total number of rows in orders

```
SELECT COUNT(*) FROM orders;
```

3. Return n random records from users

```
SELECT * FROM users  
ORDER BY RAND()  
LIMIT 5; -- replace 5 with any n
```

4. Find null values in orders

```
SELECT * FROM orders  
WHERE delivery_rating IS NULL  
OR restaurant_rating IS NULL  
OR partner_id IS NULL;
```

5. Find number of orders placed by each customer

```
SELECT u.name, COUNT(o.order_id) AS total_orders  
FROM users u  
LEFT JOIN orders o ON u.user_id = o.user_id  
GROUP BY u.user_id, u.name  
ORDER BY total_orders DESC;
```

6. Find restaurant with most number of menu items

```
SELECT r.r_name, COUNT(m.f_id) AS total_items  
FROM restaurants r  
JOIN menu m ON r.r_id = m.r_id  
GROUP BY r.r_id, r.r_name  
ORDER BY total_items DESC  
LIMIT 1;
```

7. Find number of votes and avg restaurant rating for all restaurants

```
SELECT r.r_name,  
COUNT(o.order_id) AS total_votes,  
ROUND(AVG(o.restaurant_rating), 2) AS avg_rating  
FROM restaurants r  
JOIN orders o ON r.r_id = o.r_id  
GROUP BY r.r_id, r.r_name  
ORDER BY avg_rating DESC;
```

8. Find food item sold at most number of restaurants

```
SELECT f.f_name, COUNT(DISTINCT m.r_id) AS num_restaurants  
FROM food f
```

```

JOIN menu m ON f.f_id = m.f_id
GROUP BY f.f_id, f.f_name
ORDER BY num_restaurants DESC
LIMIT 1;

```

9. Find restaurant with max revenue in a given month

```

SELECT r.r_name, SUM(o.amount) AS monthly_revenue
FROM restaurants r
JOIN orders o ON r.r_id = o.r_id
WHERE MONTH(o.date) = 6 AND YEAR(o.date) = 2023 -- change month/year as needed
GROUP BY r.r_id, r.r_name
ORDER BY monthly_revenue DESC
LIMIT 1;

```

10. Find restaurants with sales > X

```

SELECT r.r_name, SUM(o.amount) AS total_sales
FROM restaurants r
JOIN orders o ON r.r_id = o.r_id
GROUP BY r.r_id, r.r_name
HAVING SUM(o.amount) > 1000; -- replace 1000 with X

```

11. Find customers who have never ordered

```

SELECT * FROM users
WHERE user_id NOT IN (
    SELECT DISTINCT user_id FROM orders
);

```

12. Show order details of a particular customer in a given date range

```

SELECT o.order_id, f.f_name, o.amount, o.date
FROM orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN food f ON od.f_id = f.f_id
WHERE o.user_id = 1 -- replace with actual user_id
AND o.date BETWEEN '2023-01-01' AND '2023-06-30';

```

13. Find favorite food of each customer (most ordered food per customer)

```

SELECT u.name, f.f_name, COUNT(*) AS frequency
FROM users u
JOIN orders o ON u.user_id = o.user_id
JOIN order_details od ON o.order_id = od.order_id
JOIN food f ON od.f_id = f.f_id
GROUP BY u.user_id, od.f_id
HAVING COUNT(*) = (
    SELECT MAX(cnt) FROM (
        SELECT o2.user_id, od2.f_id, COUNT(*) AS cnt
        FROM orders o2
        JOIN order_details od2 ON o2.order_id = od2.order_id
        WHERE o2.user_id = u.user_id
        GROUP BY o2.user_id, od2.f_id
    ) t
);

```

14. Find most costly restaurants (avg price per dish)

```
SELECT r.r_name, ROUND(AVG(m.price), 2) AS avg_price_per_dish
FROM restaurants r
JOIN menu m ON r.r_id = m.r_id
GROUP BY r.r_id, r.r_name
ORDER BY avg_price_per_dish DESC;
```

15. Find delivery partner compensation

```
SELECT dp.partner_name,
       COUNT(o.order_id) AS num_deliveries,
       ROUND(AVG(o.delivery_rating), 2) AS avg_rating,
       (COUNT(o.order_id) * 100) + (1000 * AVG(o.delivery_rating)) AS compensation
FROM delivery_partner dp
JOIN orders o ON dp.partner_id = o.partner_id
GROUP BY dp.partner_id, dp.partner_name
ORDER BY compensation DESC;
```

16. Find revenue per month for a particular restaurant

```
SELECT MONTH(o.date) AS month,
       YEAR(o.date) AS year,
       SUM(o.amount) AS monthly_revenue
FROM orders o
WHERE o.r_id = 1 -- replace with actual r_id
GROUP BY YEAR(o.date), MONTH(o.date)
ORDER BY year, month;
```

17. Find correlation between delivery_time and total rating

```
-- MySQL doesn't have CORR() built-in, calculate manually
SELECT
  (COUNT(*) * SUM(delivery_time * (delivery_rating + restaurant_rating))
   - SUM(delivery_time) * SUM(delivery_rating + restaurant_rating)) /
  (SQRT(COUNT(*) * SUM(delivery_time * delivery_time) - SUM(delivery_time) * SUM(delivery_time))
   * SQRT(COUNT(*) * SUM((delivery_rating + restaurant_rating) * (delivery_rating + restaurant_rating))
   - SUM(delivery_rating + restaurant_rating) * SUM(delivery_rating + restaurant_rating))) AS correlation
FROM orders
WHERE delivery_time IS NOT NULL
AND delivery_rating IS NOT NULL
AND restaurant_rating IS NOT NULL;
```

18. Find correlation between number of orders and avg price per restaurant

```
SELECT r.r_name,
       COUNT(o.order_id) AS num_orders,
       ROUND(AVG(m.price), 2) AS avg_price
FROM restaurants r
JOIN orders o ON r.r_id = o.r_id
JOIN menu m ON r.r_id = m.r_id
GROUP BY r.r_id, r.r_name;
-- Then apply correlation formula on num_orders and avg_price similarly to Q17
```

19. Find all veg restaurants (serve ONLY veg food)

```
SELECT r.r_name
```

```

FROM restaurants r
JOIN menu m ON r.r_id = m.r_id
JOIN food f ON m.f_id = f.f_id
GROUP BY r.r_id, r.r_name
HAVING MIN(f.type) = 'Veg' AND MAX(f.type) = 'Veg';
-- MIN = MAX = 'Veg' means ALL items in that restaurant are Veg

```

20. Find min and max order value for all customers

```

SELECT u.name,
       MIN(o.amount) AS min_order,
       MAX(o.amount) AS max_order
FROM users u
JOIN orders o ON u.user_id = o.user_id
GROUP BY u.user_id, u.name
ORDER BY u.name;

```

Bonus Questions

21. Top 3 most ordered food items overall

```

SELECT f.f_name, COUNT(*) AS total_orders
FROM food f
JOIN order_details od ON f.f_id = od.f_id
GROUP BY f.f_id, f.f_name
ORDER BY total_orders DESC
LIMIT 3;

```

22. Customers who ordered from more than 3 different restaurants

```

SELECT u.name, COUNT(DISTINCT o.r_id) AS num_restaurants
FROM users u
JOIN orders o ON u.user_id = o.user_id
GROUP BY u.user_id, u.name
HAVING COUNT(DISTINCT o.r_id) > 3;

```

23. Restaurant with highest average delivery rating

```

SELECT r.r_name, ROUND(AVG(o.delivery_rating), 2) AS avg_delivery_rating
FROM restaurants r
JOIN orders o ON r.r_id = o.r_id
WHERE o.delivery_rating IS NOT NULL
GROUP BY r.r_id, r.r_name
ORDER BY avg_delivery_rating DESC
LIMIT 1;

```

24. Month with highest total revenue

```

SELECT MONTH(date) AS month, YEAR(date) AS year, SUM(amount) AS total_revenue
FROM orders
GROUP BY YEAR(date), MONTH(date)
ORDER BY total_revenue DESC
LIMIT 1;

```

25. Customers whose total spending > avg spending of all customers

```
SELECT u.name, SUM(o.amount) AS total_spent
FROM users u
JOIN orders o ON u.user_id = o.user_id
GROUP BY u.user_id, u.name
HAVING SUM(o.amount) > (
    SELECT AVG(total_per_user) FROM (
        SELECT SUM(amount) AS total_per_user
        FROM orders
        GROUP BY user_id
    ) t
);
```